COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE21S2 - Data Structures and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/2026	**DATE SUBMITTED:**	09/23/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO.9.1 Tree ADT

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Task 1

main.cpp | gentree.h

HOA9p1 > Procedures > GenTree > gentree.h > AddChild(T)

```cpp
#ifndef GENTREE_H
#define GENTREE_H

template<typename T>
class Node{
    public:
        T data;
        Node<T>* firstChild;
        Node<T>* nextSibling;
        Node(T value) : data(value), firstChild(nullptr), nextSibling(nullptr) {}
        Node<T>* AddChild(T childValue);

};

template<typename T>
Node<T>* Node<T>::AddChild(T childValue){
    Node<T>* newNode = new Node<T>(childValue);

    if (this->firstChild == nullptr){
        this->firstChild = newNode;

    }else{
        Node<T>* temp = this->firstChild;
        while (temp->nextSibling != nullptr){
            temp = temp->nextSibling;
        }
        temp->nextSibling = newNode;
    }
    return newNode;
}



#endif //GENTREE_H
```

Figure 1. Code Snippet of the gentree.h

main.cpp ● gentree.h ×

HOA9p1 > Procedures > GenTree > main.cpp > DisplayTree<T>(Node<T>*, int)

```cpp
#include <iostream>
#include "gentree.h"

template <typename T>
void DisplayTree(Node<T>* root, int depth = 0) {
    if (root == nullptr) return;

    for (int i = 0; i < depth; ++i) {
        std::cout << "  ";
    }
    std::cout << "- " << root->data << std::endl;

    DisplayTree(root->firstChild, depth + 1);

    DisplayTree(root->nextSibling, depth);
}

Node<char>* BuildTree() {
    // Allocate using the class constructor
    Node<char>* root = new Node<char>('A');
    // Linking the first layer of the child nodes

    Node<char>* B = root->AddChild('B');
    Node<char>* C = root->AddChild('C');
    Node<char>* D = root->AddChild('D');
    Node<char>* E = root->AddChild('E');
    Node<char>* F = root->AddChild('F');
    Node<char>* G = root->AddChild('G');

    //Subtrees by using AddChild
    Node<char>* H = D->AddChild('H');

    Node<char>* I = E->AddChild('I');
    Node<char>* J = E->AddChild('J');
    Node<char>* P = J->AddChild('P');
    Node<char>* Q = J->AddChild('Q');

    Node<char>* K = F->AddChild('K');
    Node<char>* L = F->AddChild('L');
    Node<char>* M = F->AddChild('M');

    Node<char>* N = G->AddChild('N');

    return root;
}
```

Figure 2. Code Snippet of main.cpp

```
46
47  int main() {
48
49      Node<char>* root = BuildTree();
50
51      std::cout << "\n General tree: " << std::endl;
52      DisplayTree(root);
53
54      return 0;
55  }
56
```

Figure 3. Code Snippet of the int main in main.cpp

```
 General tree:
- A
  - B
  - C
  - D
    - H
  - E
    - I
    - J
      - P
      - Q
  - F
    - K
    - L
    - M
  - G
    - N
```

Figure 4. Snippet of the output in the terminal

ILO A: Task 2

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2

I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	2
P	0	3
Q	0	3

ILO B: Task 3.1

Pre-Order	[A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N]
In-Order	[B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A]
Post-Order	[B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A]

ILO B: Task 3.2

Pre-Order	
Code:	Console: Pre Order: A B C D H E I J P Q F K L M G N

```
template<typename T>
void Node<T>::preOrder(){

    std::cout << data << " ";

    if(firstChild != nullptr){
        firstChild->preOrder();
    }

    if(nextSibling != nullptr){
        nextSibling->preOrder();
    }
}
```

Observations: From my observation the output of the first one and the second which is the program itself came to be the same while following the node->left->right.

In-Order

Code:

```
template<typename T>
void Node<T>::inOrder(){

    if(firstChild != nullptr){
        firstChild->inOrder();
    }

    std::cout << data << " ";

    if(nextSibling != nullptr){
        nextSibling->inOrder();
    }
}
```

Console:

```
In Order: B C H D I P Q J E K L M F N G A
```

Observations: The second output and the output of my guess came to reach the same arrangement while following the algorithm of left->node->right.

Post-Order

Code:

Console:

```
template<typename T>
void Node<T>::postOrder(){

    if(firstChild != nullptr){
        firstChild->postOrder();
    }

    if(nextSibling != nullptr){
        nextSibling->postOrder();
    }

    std::cout << data << " ";
}
```

```
Post Order: B C H D I P Q J E K L M F N G A
```

Observations: This time the output from the first one and for this one is also the same, from what I noticed the in-order and post-order follows the same sequence despite post-order taking a left->right->node.

ILO B: Task 3.3

```
Pre Order Searching Q:
Q was found!

In Order Searching H:
H was found!

Post-Order Searching M:
M was found!
```

Figure 5. Snippet of the output in the terminal

```
template<typename T>
Node<T>* Node<T>::SearchPreOrder(T key){

    if (this->data == key) {
        return this;
    }

    Node<T>* child = this->firstChild;

    while (child != nullptr) {

        Node<T>* result = child->SearchPreOrder(key);

        if (result != nullptr) {
            return result;
        }

        child = child->nextSibling;
    }

    return nullptr;
}
```

Figure 6. Code Snippet of the SearchPreOrder function in the header

```cpp
template<typename T>
Node<T>* Node<T>::SearchInOrder(T key) {

    Node<T>* child = this->firstChild;

    if (child != nullptr) {

        Node<T>* result = child->SearchInOrder(key);
        if (result != nullptr) {
            return result;
        }

        if (this->data == key) {
            return this;
        }

        child = child->nextSibling;

        while (child != nullptr) {
            result = child->SearchInOrder(key);
            if (result != nullptr) {
                return result;
            }
            child = child->nextSibling;
        }
    }else {

        if (this->data == key) {
            return this;
        }
    }

    return nullptr;
}
```

Figure 7. Code Snippet of the SearchInOrder function in the header

```cpp
template<typename T>
Node<T>* Node<T>::SearchPostOrder(T key) {

    Node<T>* child = this->firstChild;

    while (child != nullptr) {

        Node<T>* result = child->SearchPostOrder(key);

        if (result != nullptr) {
            return result;
        }

        child = child->nextSibling;
    }

    if (this->data == key) {
        return this;
    }

    return nullptr;
}
```

Figure 8. Code Snippet of the SearchPostOrder function in the header

```cpp
template<typename T>
void Node<T>::FindData(std::string choice, T key) {

    Node<T>* result = nullptr;

    if (choice == "PreOrder") {
        result = this->SearchPreOrder(key);

    } else if (choice == "PostOrder") {
        result = this->SearchPostOrder(key);

    } else if (choice == "InOrder") {
        result = this->SearchInOrder(key);
    }

    if (result != nullptr) {
        std::cout << key << " was found!" << std::endl;
    }
}
```

Figure 9. Code Snippet of the FindData function in the header

ILO B: Task 3.4

```
New General Tree:
- A
  - B
  - C
  - D
    - H
  - E
    - I
    - J
      - P
      - Q
  - F
    - K
    - L
    - M
  - G
    - N
    - O
Searching for O:
O was found!
```

Figure 10. Snippet of the output inside the terminal

```
template<typename T>
void Node<T>::AddNodeO() {

    Node<T>* G = this->SearchPreOrder('G');

    if (G != nullptr) {
        G->AddChild('O');
    }
}
```

Figure 11. Code Snippet of the function inside the header file

```
std::cout << "Adding Node 'O' " << std::endl;
root->AddNodeO();

std::cout << "Pre Order after adding O: ";
root->preOrder();
std::cout << std::endl;

std::cout << "New General Tree: " << std::endl;
DisplayTree(root);

std::cout<<"Searching for O: "<<std::endl;
root->FindData("PreOrder", 'O');
std::cout<<std::endl;
```

Figure 12. Code Snippet of main for Adding and finding 0

From my observation, the O was added under G, but because it was larger than N it was placed to the right of it.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

main.cpp

```
OA9p1 > Procedures > DEMO > main.cpp > mair
#include <iostream>
#include "BST.h"

int main(){
    //create the root node:
    BinarySearchTree<int> tree(2);

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);


    std::cout<<"preOrder: ";
    tree.preOrder();
    std::cout<<std::endl;

    std::cout<<"inOrder: ";
    tree.inOrder();
    std::cout<<std::endl;

    std::cout<<"PostOrder: ";
    tree.postOrder();
    std::cout<<std::endl;


    return 0;
}
```

Figure 1. Code Snippet of binary search tree insertion

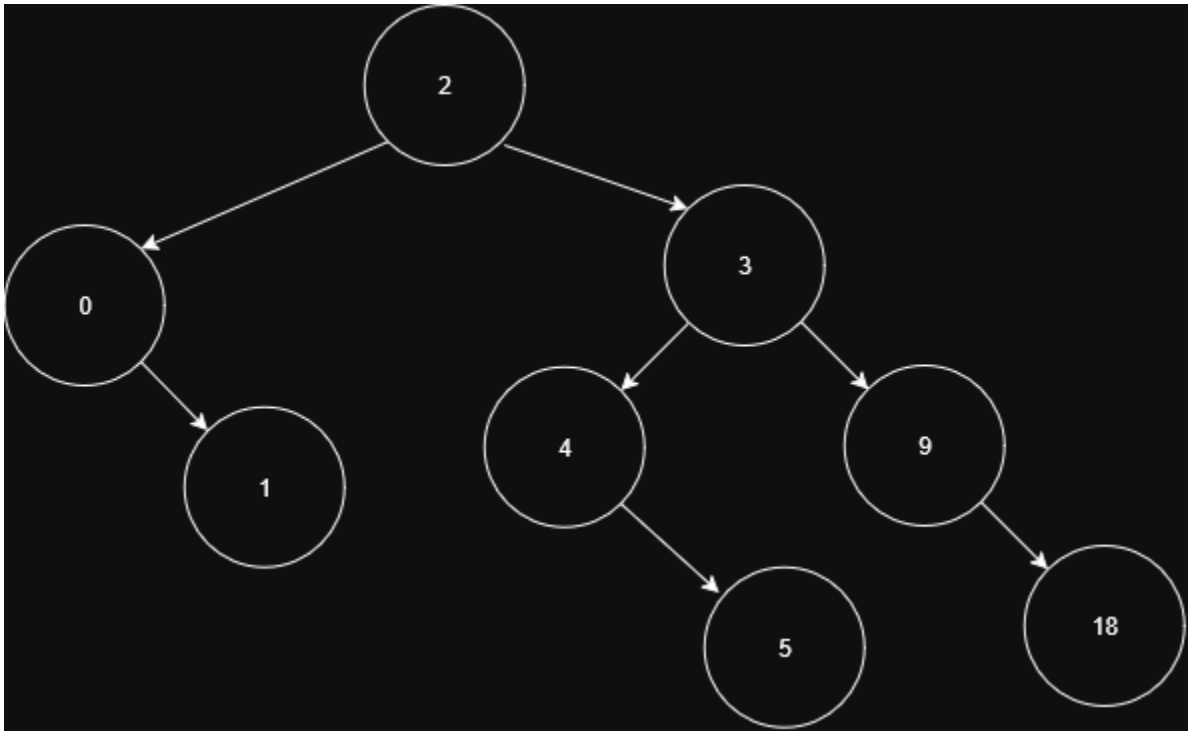


Figure 2. Snippet of Tree Diagram

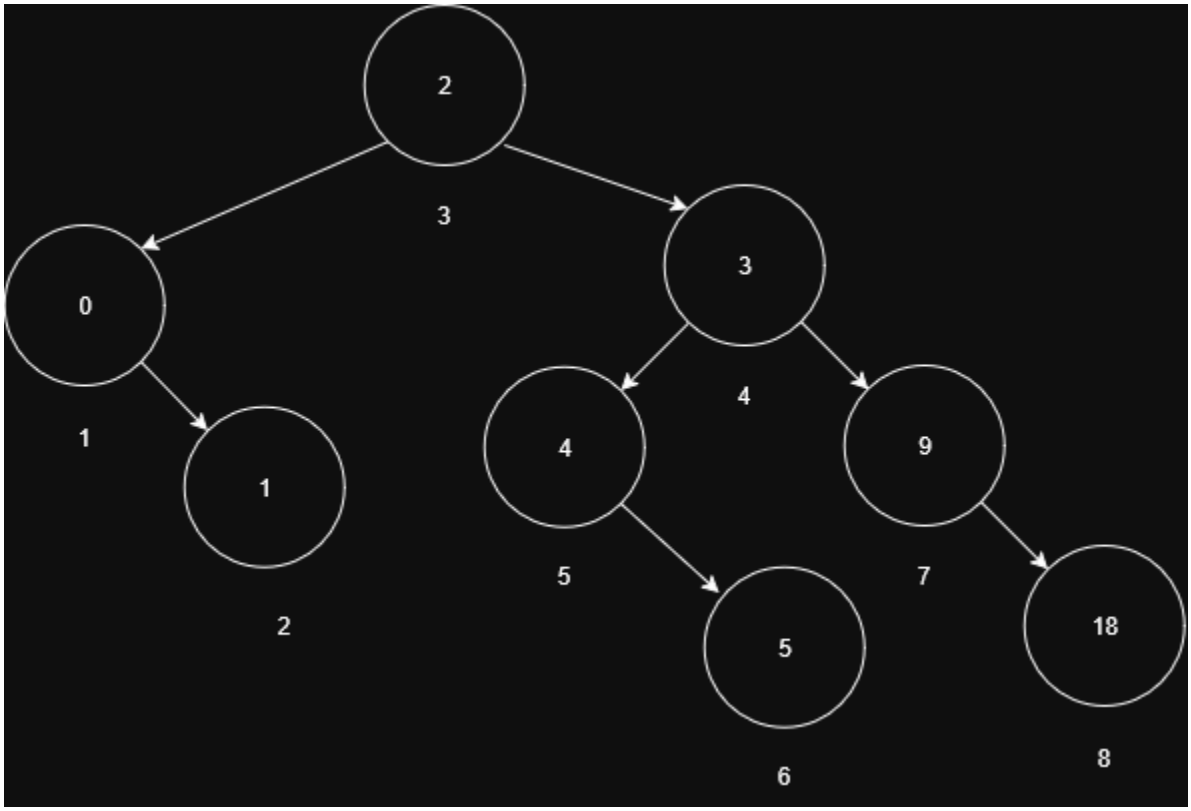


Figure 2. Snippet of inOrder Diagram

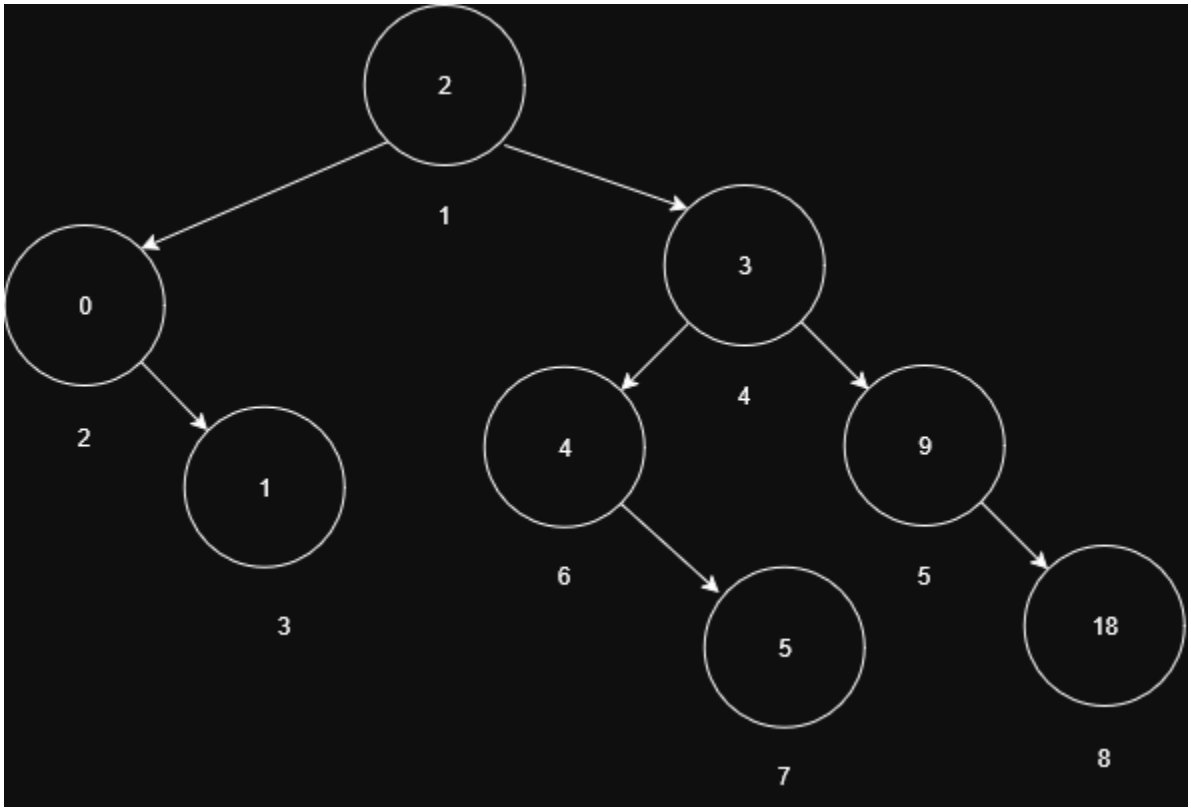


Figure 3. Snippet of preOrder Diagram

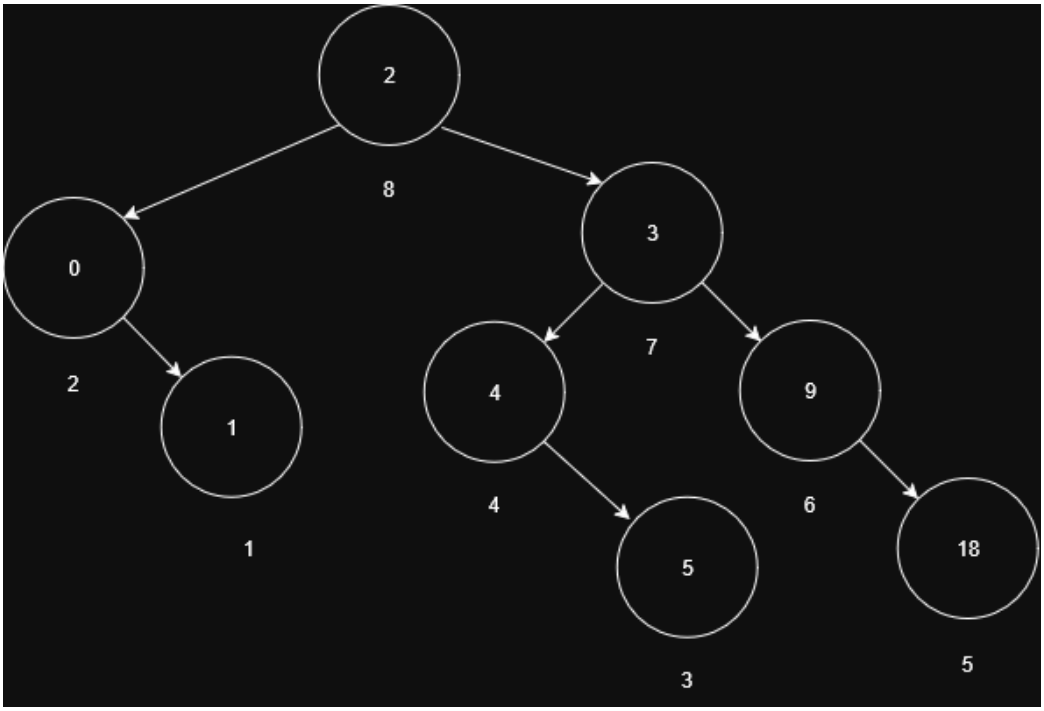


Figure 4. Snippet of postOrder Diagram

```
template<typename T>
void BinarySearchTree<T>::inOrder(){
    //go to left
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<< key << " ";

    //go to right
    if(right != nullptr){
        right->inOrder();
    }

}
```

Figure 5. Code Snippet of the inOrder traversal

Pre-Order Traversal

```
template<typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<< key <<" ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right != nullptr){
        right->preOrder();
    }
}
```

Figure 6. Code Snippet of the function preOrder traversal

```
preOrder: 2 0 1 3 9 4 5 18
```

Figure 7. Code Snippet of the output of preOrder traversal

From my observation, the output is the same from the manual traversal that was indicated in Figure 3 which starts as 2 and ends with 18 after completely traversing the left side then continuing with the right until the end.

```
template<typename T>
void BinarySearchTree<T>::postOrder(){
    //go to left
    if(left != nullptr){
        left->postOrder();
    }

    //go to right

    if(right != nullptr){
        right->postOrder();
    }

    std::cout<< key << " ";

}
```

Figure 8. Code Snippet of the function postOrder traversal

```
PostOrder: 1 0 5 4 18 9 3 2
```

Figure 9. Snippet of the output of the postOrder traversal

From my observation the output did reach the same front and end, starting with the right to the left until the node which was 2.

Conclusion
In this hands-on activity we delved deeper into the concepts of trees such as the other three trees such as

general tree, binary tree and binary search tree. Their difference is mostly noticeable on the maximum children they can have per node, if the binary tree and binary search tree is strictly limited to a maximum of 2, the general tree can have as many as possible. Apart from that is their ordering rules, general tree does not have any ordering rules, while the binary tree, the left and right are just positions, but binary search tree has a strict ordering rule depending on the type of traversal.

For the procedure, I used AI for researching and skimming multiple sources for different implementations for the general tree. Apart from that, the height and depth can be simply differentiated using the height as down-up while the depth is top-down. Moreover, the difference from the implementation in the general search tree compared to the binary search tree was entirely different with the general tree being implemented like a linked list, with the traversals being different as well depending on preference.

For the supplementary activity, using the binary search tree, the three traversals such as pre-order (root->left->right), in-order(left->root->right) and lastly, the post-order (left->right->root). These have different ways of traversing and arranging the tree for example in the figure 2 to 4 of the supplementary where it was clear that they all follow different rules and what they take first and take last. Overall, the activity was fairly simple in understanding the logic but the syntax was still slightly confusing in actually implementing the different types of trees itself rather than the logic of the traversal.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and Research**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Claude AI for skimming the internet of a general tree implementation to be used for the procedure ILO A.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.